In-context Learning of Evolving Data Streams with Tabular Foundational Models

Afonso Lourenço
GECAD, ISEP, Polytechnic of Porto
Porto, Portugal

João Gama
INESC-TEC, FEP, University of Porto
Porto, Portugal

Eric P. Xing
Carnegie Mellon University
Mohamed bin Zayed University of AI
Pittsburgh, USA

Goreti Marreiros
GECAD, ISEP, Polytechnic of Porto
Porto, Portugal

Abstract

State-of-the-art data stream mining has long drawn from ensembles of the Very Fast Decision Tree, a seminal algorithm honored with the 2015 KDD Test-of-Time Award. However, the emergence of large tabular models, i.e., transformers designed for structured numerical data, marks a significant paradigm shift. These models move beyond traditional weight updates, instead employing in-context learning through prompt tuning. By using on-the-fly sketches to summarize unbounded streaming data, one can feed this information into a pre-trained model for efficient processing. This work bridges advancements from both areas, highlighting how transformers' implicit meta-learning abilities, pre-training on drifting natural data, and reliance on context optimization directly address the core challenges of adaptive learning in dynamic environments. Exploring real-time model adaptation, this research demonstrates that TabPFN, coupled with a simple sliding memory strategy, consistently outperforms ensembles of Hoeffding trees, such as Adaptive Random Forest, and Streaming Random Patches, across all non-stationary benchmarks.

CCS Concepts

• **Computing methodologies** → **Online learning settings**; **Learning under covariate shift**.

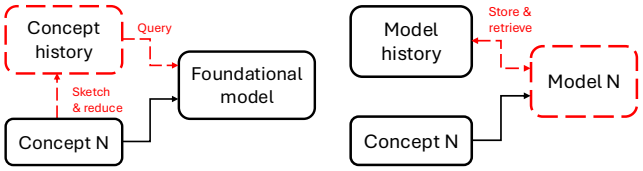
Keywords

Concept drift, data stream mining, foundational model

ACM Reference Format:
Afonso Lourenço, João Gama, Eric P. Xing, and Goreti Marreiros. 2026. In-context Learning of Evolving Data Streams with Tabular Foundational Models. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3770854.3780305

1 Introduction

Data stream mining is an area in machine learning where the inference and training of the algorithm are performed in real-time,



(a) In-context stream mining **(b) Conventional stream mining**

Figure 1: A new paradigm

dealing with large volumes of ever-evolving tabular data [21]. Differently to batch learning where all training data necessary to induce a model is available, data streams incrementally arrive at any time. Thus, making a model easily outdated due to the occurrence of concept drifts, i.e., distribution changes over time [23]. To circumvent these challenges, many stream mining algorithms have been proposed with different requirements being imposed: (1) be ready to predict and update the model at any point, and in limited time; (2) process an example at a time, inspecting only once with a limited sized model; (3) be able to adapt to change.

Decision trees. In fulfilling these requirements, the current state-of-art for supervised learning in tabular data streams has long been incremental decision trees (IDTs), whose success can be attributed to two key factors: approximation-based splitting techniques [38] and effective adaptation strategies [48]. Approximation-based splitting involves updating statistical summaries of entropy-based metrics, e.g. information gain, and determining whether the observed utility of a split is statistically close to its true utility when the data distribution is unknown, e.g. via the Hoeffding bound [19]. However, as tree grows from the root node, the descendant nodes subsequently get fixed to covering particular sub-spaces of the space covered by their parent node. To address this, adaptation strategies allow to trigger the forgetting of old knowledge, pruning the affected parent nodes, instead of a complete replacement of the base learner [6]. Alternatively, multiple trees can be combined into a high diversity ensemble, which naturally develops decision boundaries that locally adapt to changes in the data distribution [26, 34]. Upon a concept drift, the diversity among trees can be exploited as some local minima are less affected than others. Affected components can be dynamically deleted, added, and combined with a voting scheme, weighting learners based on their past performance. Moreover, meta-learning [3] and model-based clustering

This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2258-5/2026/08
https://doi.org/10.1145/3770854.3780305

[63] allows to more explicitly model history mechanisms to retain a pool of previous active and inactive concepts.

Lack of autonomy. Regardless of how sophisticated these strategies are, these still rely on human expertise to select the most suitable algorithm, or determine the optimal configuration for a given algorithm. Indeed, combining all these building blocks in a single solution normally yields more inductive biases to assume and hyperparameters to tune. Consequently, as new concepts emerge, the performance curves of different algorithm settings cross. To address this, one can simply build heterogeneous ensembles with different algorithms, whose weights are dynamically selected based on recent performance [56]. Alternatively, one can calculate metafeatures over sliding windows to build a meta-model that predicts the best algorithm [55], the best drift detector [1], or the most suitable uncertainty threshold for active learning strategies [42], in the next interval of unseen data.

Foundational models (FMs). Conversely, the disruptive success and knowledge-modeling capabilities of foundational models [10] brings the opportunity to tap into prior experience, to the point that most of the required learning has already been done beforehand. While the aforementioned streaming methods follow the traditional two-stage methodology, i.e., first optimizing the model with several tuning iterations over a sliding window to then select the best algorithm or hyperparameter configuration for the next window, FMs allow to instantaneously deploy a model, bypassing the computational and temporal costs of this traditional process. By pretraining on extremely large corpora of datasets, FMs develop common sense inductive biases from the infinite amounts of prior learning experiences. Thus, driving emergent abilities that improve with scale, such as few-shot in-context learning (ICL), which allows models to perform new tasks at inference time by conditioning on a set of input-output examples, without any parameter updates [10].

Textual tabular prompts. Consequently, this ICL ability pushed a new research paradigm, focused on adapting pre-trained models through prompt tuning rather than adjusting model weights, by discrete search with natural language, or optimizing the model's embedding space directly. Within this paradigm and the emerging possibilities of multimodal applications, various efforts were made for flexible textual prompts of structured tabular data with heterogeneous feature spaces and mixed-type columns [20]. As language-based FMs' capabilities on table understanding were validated, new efforts went towards directly applying them for supervised learning of tabular data, i.e. predicting an unseen sample by continuing its textual description on the target column [17, 29]. However, despite the potential of these advancements to perform semantically-driven drift detection and model adaptation, several limitations are expected to persist in the near term hindering their application for data stream mining: limited context windows, quadratic complexity costs, struggle with autoregressively understanding continuous variables [52, 53], and sensitivity to unexpected characters [65].

Large Tabular Models (LTMs). To alleviate all these limitations, one can shift to designing a dedicated architecture for numeric data from scratch, completely pre-trained on a wide range of tabular datasets with different data distributions, allowing it to learn relevant and general meta-features. These are referred to as LTMs [54]. While it has been known for a while that transformers can handle variable length input sequences in order to perform pretraining and cross-table knowledge transfer [60], it has been recently demonstrated that finetuning from a pretrained tabular transformer [27, 51, 61] is superior to training tabular transformers from scratch [66]. Following these findings, various LTMs pretrained on synthetic tabular data have been proposed for classification [9, 31, 64]. For instance, the original TabPFN [30] is pre-trained with a 12-layer Transformer for 18000 batches of 512 synthetically generated datasets generated using randomly initialized neural networks to impose the diverse interrelations that exist among the features of realistic tabular datasets, e.g. causal relationships and feature importance [45]. While this training step is moderately expensive, requiring a total of 20 hours on 8 GPUs (Nvidia RTX 2080 Ti), it is done offline, in advance, and only once, as part of the algorithm development. Then, in deployment TabPFN performs instant classification without fine-tuning, adapting to unseen datasets in a single forward pass at inference time by using various training examples as context, analogously to how LLMs use the preceding tokens [10]. However, in contrast to LLMs, LTMs like TabPFN allow scaling to much larger datasets, saving tuning resources during deployment, while providing consistent behavior supported by traditional statistical learning.

This work. Building on these advancements, this work proposes a new paradigm, using on-the-fly techniques to summarize unbounded data streams before feeding them to LTMs. While typical streaming methods rely on adapting the model parameters to deal with non-stationary data distributions, this new paradigm uses input data as learnable parameters without changing model parameters, decoupling the design of the learning algorithm from the foundational model. The main idea is to transform big data into a sketch whose size and running time has little or no dependency on the size of the data stream, as illustrated in Figure 1. The learning algorithm is then applied to this dynamically contextualized sketch, with the main challenge of balancing the trade-off between sketch size and information loss. Thus, shifting from in-weights learning to in-context learning. The main contributions are outlined as follows:

- Introduce a novel in-context learning paradigm for tabular data streams, formalizing how the distributional qualities of the pre-training data and a mesa-optimization architectural bias address the core challenges of adaptive learning.
- Extend the popular TabPFN transformer [30] with a new inference-time sketching mechanism, without altering its pretrained weights. Inspired by a bayesian and frequentistic perspective, we introduce a bias-variance decomposition of prediction error, proving variance reduction through more in-context examples, and addressing drifting context-induced biases via a dual-memory FIFO mechanism that maintains inter-class balance and captures intra-class variability through long- and short-term buffers.

2 In-context stream learning

ICL requires a model to implicitly construct a map from in-context examples to a predictor of a new concept on which it has not been previously trained, without any updates to the model's parameters themselves. Indeed, it has been show empirically that transformers with fixed parameters can learn linear functions comparable to the

optimal least squares estimator, by encoding smaller models in their activations, and updating these implicit models as new examples appear in the context on the fly [59]. Furthermore, it has been demonstrated that transformers implement more complex function classes implicitly, i.e., sparse linear functions, two-layer neural networks, and decision trees, with performance that matches or exceeds task-specific learning algorithms [44]. However, it remains unclear to what extent these models are able to learn new tasks from in-context examples alone as opposed to indexing into a vast set of known tasks from the training data. In this regard, it has been hypothesized that this emergent behavior of transformers is driven by both the distributional qualities of the training data itself [12] and an architectural bias where the internal structure of the model ends up performing mesa-optimization [4].

Natural data hypothesis. The data hypothesis was inspired by the observation that many natural data sources differ from typical batch supervised datasets due to a few notable features: (1) natural data is temporally bursty, i.e. a given concept may have a distribution that is not uniform across time, instead tending to appear in clusters [2]; (2) the meaning of these concepts in natural data is often dynamic rather than fixed, usually in a context-dependent way; (3) the marginal distribution across entities is highly skewed, with large numbers of rarely occurring classes [12]. Indeed, these are the properties of data streams that motivate the development of incremental learning algorithms. While batch i.i.d. training typically consists of item classes that reoccur with uniform regularity, and input-output mappings that are fixed throughout training, data stream mining implies training on sequences of data with drifting concepts, akin to having many labels per item [49], a few highly reoccurring concepts [3], and a large number of outliers that reoccur much more rarely, thus excluded from the concept forming process due to their disproportionately less likelihood to occur multiple times within a given context window [16].

Mesa-optimization hypothesis. The algorithm hypothesis identifies this emergent behavior as stemming from an architectural bias where the internal structure of transformers ends up performing mesa-optimization [4], which doesn't happen for other recurrent models, like LSTMs and RNNs, even if matched on number of parameters [12]. Mesa-optimization occurs when a base optimizer is searching for algorithms to solve some problem and finds a model that is itself a mesa-optimizer [32]. Unlike the base objective, the mesa-objective is not specified directly by the programmers. Instead, a learned process running within the forward pass constructs an internal mesa-objective, and its corresponding solution through mesa-optimization, because it is instrumentally useful for solving the given task. While meta means to go above, thus stepping outside to a higher logical level, mesa means going to the other direction, to go down into the content narrowing the details into smaller and smaller pieces [13]. Many reasons have been identified for this phenomenon. Firstly, the base optimizer's incentive to find a compressed policy creates a bias towards simple solutions with low description length. Moreover, the transformer's statefulness favors mesa-optimization by decreasing the implicit penalty on time complexity imposed by enforcing all computations to end when the learned algorithm produces its output. And, the ability of the learned algorithm to save and refer back to previous state enables the caching of intermediate results. These factors, when combined with heavy pre-training on diverse tasks that are likely to be completely novel, incentivize the base optimizer to expend optimization power in learning how to adjust to a new task immediately in order to perform consistently in an diverse environment. Figure 2 illustrates this ability.

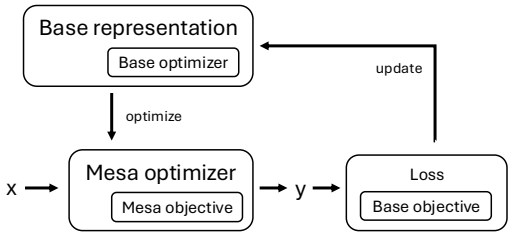


Figure 2: ICL as a mesa-optimization capability

Implicit meta-learning. While breaking things down into bits is a mesa process, the model always returns to rebuild and re-map the actual details. In going mesa, all generalizations are deconstructed which allows to reconstruct the input-output mapping in a more resourceful manner. In this regard, training a model to perform ICL can also be viewed as an instance of the learning-to-learn or meta-learning paradigm, with learning that takes place on various time scales, fast and slow, comprising both mesa- and meta-processes. Examples include approximating an implicit model selection procedure competitive with empirical risk minimization [36], a gradient-based few-shot learning within its forward pass [58] or a kernel-based sequence model [35]. However, in these cases, researchers explicitly designed the meta-training regime to incentivize ICL, whereas in the case of transformers, the capacity for ICL is emergent. Neither the model's transformer architecture nor its learning objective are explicitly designed with ICL in mind.

3 Large tabular model

Since the pre-training stage does not represent the added value of this work, we rely on the popular TabPFN transformer-based model [30, 31], relying on its original dataset generation and architecture, only performing minor modifications on the inference time procedure. At the core of TabPFN is the self-attention mechanism, interchanging information among different positions within a sequence [57]. In self-attention, each element in the input sequence of length T is linearly projected to produce d-dimensional queries, keys, and values, each represented by $Q, K, V \in \mathbb{R}^{T\times d}$. The attention matrix is computed as $\text{softmax}(QK^T/\sqrt{d})$ where softmax is applied row-wise. It is then multiplied to the value matrix to produce the output of the layer $O = \text{softmax}(QK^T/\sqrt{d})V$. If self-attention is computed in this manner, it is more precisely called bi-directional self-attention since every token can attend to all the other tokens in both directions. However, TabPFN relies on causal attention layers typical of a standard decoder-only Transformer architecture [37], making it possible to process a streaming sequence where the tokens become available one at a time [35]. The complete architecture is given in Figure 3.

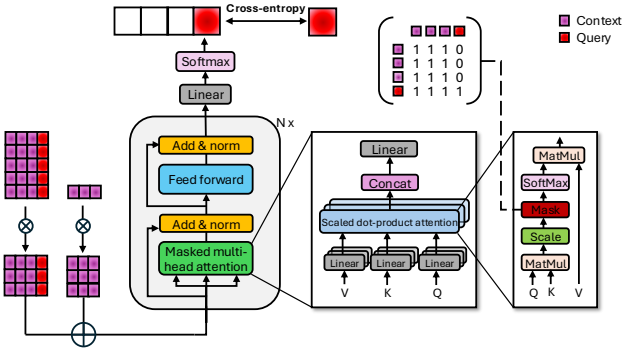


Figure 3: Architecture and training procedure of the foundational model, where ⊗ represents features and float targets being separately encoded using a linear layer, and the embedding is pushed through a decoder-only transformer model

Architecture. Each instance's feature vector is treated as a set-valued input to exploit permutation invariance and represented as a token with dimension d_{token}, allowing token representations to attend to each other for a fixed number of features d_f. Firstly, the creation of in-context tokens $H \in \mathbb{R}^{|S| \times d_{\text{token}}}$ is perfomed by embedding the features and classes linearly using weights $W_x \in \mathbb{R}^{d_f \times d_{\text{token}}}$ and $w_y \in \mathbb{R}^{d_{\text{token}}}$ respectively. Then, the embedded tokens are pushed through the standard decoder-only Transformer [37]. Each layer in the transformer autoregressively maps the matrix H^l to a sequence H^{l+1}, processing each collumn h_i^l of H^l in parallel. Each layer first computes a self attention and then applies a feedforward transformation. However, with a special attention mask, where in-context tokens only attend to each other, with no attention to the single query instance, as illustrated in Figure 4. Instead, the target to the held-out query instance is used as a label, minimizing the cross-entropy loss, which approximates the true Bayesian posterior predictive distribution [45]. This procedure results in a computational cost of a forward pass scaling as $O(|S|^2)$.

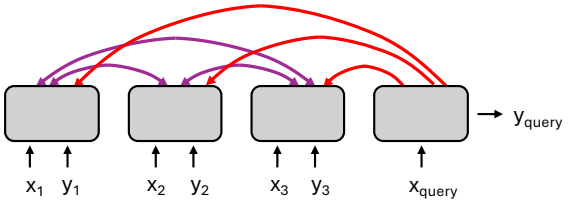


Figure 4: Special attention mask

Learning. To prior-fit the network as part of the algorithm development, TabPFN repeatedly samples synthetic realistic datasets. These are generated by carefully crafting correlations blockwise, imposing causal relationships between features, and tuning the magnitude of weights for variety in the feature importance, using Bayesian Neural Networks and Structural Causal Models with random characteristics and randomly initialized weights [30, 59]. In this regard, to better understand and quantify the uncertainty in the process of learning a LTM, one can take both a bayesian [24, 62] and a frequentistic perspective [46].

Bayesian perspective. In-context learning can be viewed as approximate Bayesian inference, whether implicitly [62] or explicitly [24]. It has be shown that a Transformer directly fits the posterior predictive distribution (PPD), with a hypothesis latent concept ϕ which could parametrize the transitions of a Hidden Markov model [5], or a space of structural causal models. In this framework, the prior defines a space of hypotheses Φ on the relationship of a set of inputs x to the output labels y. Each hypothesis $\phi \in \Phi$ can be seen as a mechanism that generates a data distribution from which one can draw context data with observed labels $x_1, y_1, \ldots, x_n, y_n$, denoted as c_{txt}, and an unlabeled query sample x_q, as illustrated in Figure 5.

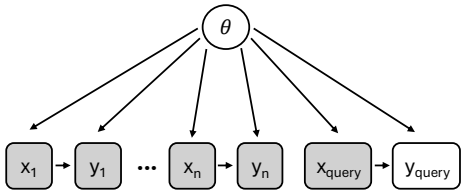


Figure 5: ICL as Bayesian inference

The PPD specifies the distribution of the label $p(y_q|x_q, c_{txt})$ and can be obtained by integration over the space of hypotheses Φ, where the weight of a hypothesis $\phi \in \Phi$ is determined by the posterior probability $p(\phi|c_{txt})$ given the context data. Considering the unnormalized posterior, which corresponds to the prior probability $p(\phi)$ and the likelihood $p(c_{txt}|\phi)$ of the training data given ϕ, we have:

$$p(y_q|c_{txt}, x_q) = \int_{\Phi} p(y_q|x_q, \phi) p(c_{txt}|\phi) p(\phi) d\phi \tag{1}$$

Frequentistic perspective. Despite contexts being sampled from a different distribution than the pretraining distribution, the asymptotic prediction error of in-context learning is optimal when the signal about the latent concept in each context example is larger than the error due to the distribution mismatch. As a more informative and representative input space is provided, the learning error decreases [62]. In this regard, one can also take a frequentistic perspective [46]. From a predictor's variance standpoint, being a pre-tuned, but untrained predictor with many hyperparameters and multi-head attention, a LTM has extremely high sensitivity to individual training samples, which translates in an increased ability to choose submodels and vanishing variance. From a bias standpoint, hyperparameters are pre-tuned to be optimal for a set of tasks defined by the prior. If this prior has large enough support and does not concentrate too much away from the true hypothesis, one can guarantee that the PPD converges to a close approximation of the true predictive distribution. Consequently, whether the LTM predictor can learn at inference time depends on its structural properties, with the optimal LTM approximation being characterized

by a Kullback-Leibler criterion [46]. To allow for accurate approximation of the conditional class probabilities, one needs sufficiently complex LTM models and prior. Practically, a LTM is trained on simulated data sets. The larger these data sets are, the more complex the PPD is approximated. The training set size can therefore be understood as a regularizer on the expected complexity of the network. Thus, while LTM implementations ensure only vanishing predictor's variance, the bias vanishes only if the provided data is somewhat localized around the test feature.

4 Context optimization

Building on the theoretical insights into how LTMs can perform in-context stream learning, we now turn to the challenge of regulating their behavior in non-stationary environments. Specifically, the objective of context optimization is to minimize bias in model predictions when faced with new data that drifts over time. This problem occurs in situations where the LTM f_θ, is applied on-the-fly to a continuous stream of real-world data. The data stream is assumed to be partitioned sequentially into non-overlapping segments, called concepts, such as C_i and C_{i+1}, each corresponding to distinct joint data distributions, $p_i(x, y)$ and $p_{i+1}(x, y)$, respectively. Within each concept, the data distribution is assumed static, however, as the data stream progresses, drift between consecutive concepts can introduce discrepancies between past and future data. For instance, if concept C_i has a skewed label distribution $p_i(y)$, and the following concept C_{i+1} has a uniform label distribution $p_{i+1}(y)$, then the predictive model must adapt to account for this shift in label distribution [47]. The goal is to select an optimal localized context l, denoted by (c_{txt}, x_q), which helps f_θ approximate the correct output y_q even for cases in the tail regions of these drifting distributions. In this case, the model's expected prediction error for a new sample (x_q, y_q) is defined by:

$$\begin{aligned}
&\mathbb{E}_l[(f_\theta(x_q \mid c_{txt}) - \mathbb{E}[y_q \mid x_q])^2] \\
&= \mathbb{E}_l[((f_\theta(x_q \mid c_{txt}) - \mathbb{E}_l[f_\theta(x_q \mid c_{txt})]) \\
&+ (\mathbb{E}_l[f_\theta(x_q \mid c_{txt})] - \mathbb{E}[y_q \mid x_q]))^2] \\
&= \mathrm{Var}[f_\theta(x_q \mid c_{txt})] + \left(\mathrm{Bias}^2[f_\theta(x_q \mid c_{txt})]\right) + \sigma^2
\end{aligned} \tag{2}$$

LTM's variance decreases with more in-context examples. Assume f_θ to be c-Lipschitz continuous with constant $c = (c_1, \ldots, c_n)$ where each c_i scales as $\delta i^{-\alpha}$, δ is a positive constant, and $\alpha > 0.5$. Then, for any two independently sampled contexts (c_{txt}, x_q) and (c'_{txt}, x_q) from an unbounded stream D, the following inequality $|f_\theta(x_q \mid (c_{txt})) - f_\theta(x_q \mid (c'_{txt}))| \leq \sum_{i=1}^{n} c_i \mathbf{1}_{\{x_i \neq x'_i\}}$ holds [24]. Applying McDiarmid's Inequality [43], for any $t > 0$, the tail probability bound is:

$$Pr(|f_\theta(x_q \mid c_{txt}) - \mathbb{E}_l[f_\theta(x_q \mid c_{txt})]| \geq t) \leq 2\exp\left(-\frac{2t^2}{|c|_2^2}\right) \tag{3}$$

where $|c|_2^2 = \sum_{i=1}^{\infty}(\delta i^{-\alpha})^2$, converging due to $\alpha > 0.5$. By the Borel-Cantelli lemma, $f_\theta(x_q \mid c_{txt})$ converges almost surely to the expected prediction $\mathbb{E}_\sim[f_\theta(x_q \mid c_{txt})]$ as $n \to \infty$ [47]. This setup implies that as more samples are added to the prompt, the model's sensitivity to small input changes diminishes, reducing error volatility.

LTM's biases inherited from in-context examples. Assume f_θ mimics gradient descent steps on examples (c_{txt}, x_q), the model's average prediction over many samples $\mathbb{E}[f_\theta(x_q \mid c_{txt})]$ almost surely converges to the true conditional expectation $\mathbb{E}[y_q \mid x_q]$ as $n \to \infty$, provided there is no drift in the data distribution [58]. However, if there is shift in the distribution, (c_{txt}, x_q) may become misaligned with the current distribution, leading to biased predictions that reflect outdated patterns. This creates a tension between retaining examples from old concepts $\{C_j\}_{j=1}^{i-1}$ just for the global representation of all classes $y \in \mathcal{Y}$, and adapting to transient local patterns representing the most up to date concept C_i, at the cost of not representing all classes.

Dual-memory. To keep representative samples of reoccurring classes, while gradually incorporating more data from recent classes, one can create a dual-memory FIFO mechanism that combines long-term inter-class balance with short-term intra-class responsiveness $c_{\text{txt}} = S_t \cup L_t$, as depicted in Figure 6. To maintain stable inter-class boundaries over time, a long-term memory $L_t \subset D_{<t}$ of size M_{long} retains temporally older samples that ensure a balanced distribution across all known classes $\mathcal{Y}$. This memory helps preserve representation from rare or fading classes, especially when their frequency $p_i(y) \ll 1$. Over time, minority classes may evolve from outlier regions toward denser areas, or even swap roles with majority classes as the stream progresses. The long-term memory ensures that the context remains representative of the most up to date global distributional structure over extended periods. Complementarily, a short-term memory $S_t \subset D_{t-M_{\text{short}}:t}$ captures the most recent samples from the stream. This memory focuses on preserving intra-class variability and recent sub-distributions $p_t(x \mid y)$. Within a single class $y \in \mathcal{Y}$, examples may cluster into temporally local sub-concepts, separated in input space due to drift. These sub-clusters may expand, shift, or disappear with time. The short-term buffer tracks these fine-grained dynamics, enabling the model to adapt to class recurrence, sub-concept emergence, and local outlier patterns.

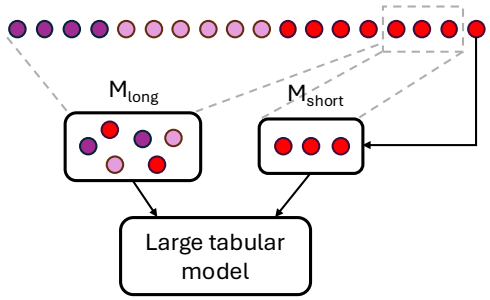


Figure 6: Short- and long-term memories

FIFO algorithm. The pseudocode is given in Algorithm 1. It begins by initializing the short-term memory S and long-term memory L, along with a class count dictionary $C[y]$ for each class y. The size of the long-term memory is computed as $M_{\text{long}} = M - M_{\text{short}}$. After a warming period T_{warm}, the algorithm enters the main loop, processing each new sample (x_t, y_t) from the data stream D_{stream}. For each new sample, the algorithm first adds it to the short-term memory S. If the short-term memory exceeds its maximum size M_{short}, the oldest sample is removed and moved to the long-term

Table 1: Prequential accuracy (*: no statistical difference), NC = No-change, MC = Majority-Class

Dataset	LTM	ARF (n=90)	BOLE (n=50)	LevBag (n=45)	SRP (n=80)	EFDT	VFDT	NC	MC
NOAA	**81.29 ± 0.26**	78.36 ± 0.20	73.45 ± 0.51	75.51 ± 0.23	78.35 ± 0.18	72.34 ± 0.43	70.86 ± 1.56	68.03	68.62
METER	**73.62 ± 0.28**	68.02 ± 0.36	67.28 ± 1.54	63.17 ± 5.78	69.69 ± 1.81	57.94 ± 2.21	52.07 ± 2.63	1.08	8.22
ELEC	**91.82 ± 0.18**	87.89 ± 0.22	90.17 ± 0.98*	85.86 ± 0.16	88.70 ± 0.57	77.96 ± 1.28	77.87 ± 1.29	85.30	57.41
RIALTO	**79.03 ± 1.06**	67.95 ± 0.36	47.27 ± 1.54	60.99 ± 5.78	75.12 ± 1.81	55.79 ± 2.21	28.68 ± 2.63	00.00	10.00
POSTURE	**59.72 ± 0.18**	59.14 ± 0.19*	48.28 ± 1.09	55.44 ± 0.69	58.14 ± 0.55	48.91 ± 0.18	52.99 ± 1.05	20.07	27.29
COVER	**95.53 ± 0.23**	91.75 ± 0.14	93.33 ± 0.59*	85.26 ± 0.24	94.54 ± 0.20*	82.38 ± 1.08	76.98 ± 2.37	95.06	48.76
POKER	**97.51 ± 0.31**	89.17 ± 0.43	80.01 ± 1.54	88.52 ± 3.12	87.58 ± 0.88	77.38 ± 4.10	77.43 ± 7.02	74.54	50.11
Average	**82.65**	77.47	71.40	73.54	78.87	67.53	62.41	49.15	38.63
Rank	**1.00**	2.86	4.57	4.29	2.71	6.00	6.57	7.43	8.43

memory L. The class count for the corresponding class of the removed sample is updated in the dictionary C. Once the long-term memory exceeds its capacity M_{long}, the algorithm identifies the most overrepresented class, $y_{\max}$, in long-term memory L. The oldest sample from this overrepresented class is then removed to maintain balance in the memory, ensuring that the system does not retain too many samples from any single class.

Algorithm 1 Naive context optimization

Input: D_{stream}, M_{short}, M_{long}, T_{warm}, LTM
$S, L \leftarrow \emptyset$
$C[y] \leftarrow 0, \forall y \in \mathcal{Y}$
$t_{\text{warm}} \leftarrow 0$
for each new sample $(x_t, y_t) \in D$ **do**
  if $t_{\text{warm}} > T_{\text{warm}}$ **then**
    $\hat{y} \leftarrow$ learn in-context$(LTM, S \cup L)$
    $A_T \leftarrow \frac{1}{T} \sum_{t=1}^{T} \mathbb{1}(\hat{y}_t = y_t)$
  end if
  $S \leftarrow S \cup \{(x_t, y_t)\}$
  if $|S| > M_{\text{short}}$ **then**
    $(x_{\text{old}}, y_{\text{old}}) \leftarrow$ oldest sample in S
    $S \leftarrow S \setminus \{(x_{\text{old}}, y_{\text{old}})\}$
    $L \leftarrow L \cup \{(x_{\text{old}}, y_{\text{old}})\}$
    $C[y_{\text{old}}] \leftarrow C[y_{\text{old}}] + 1$
  end if
  if $|L| > M_{\text{long}}$ **then**
    $y_{\max} \leftarrow \arg\max_{y \in \mathcal{Y}} C[y]$
    $(x_{\text{old}}, y_{\text{old}}) \leftarrow$ oldest sample in $\{(x, y) \in L \mid y = y_{\max}\}$
    $L \leftarrow L \setminus \{(x_{\text{old}}, y_{\text{old}})\}$
    $C[y_{\text{old}}] \leftarrow C[y_{\text{old}}] - 1$
  end if
end for

5 Experimental study

This section evaluates the performance of the proposed context optimization of a large tabular model (LTM) against state-of-art algorithms for data stream mining, namely: Adaptive Random Forest (ARF) [25], Streaming Random Patches (SRP) [26], Boosting-like Online Learning Ensemble (BOLE) [15], Leverage Bagging (LevBag) [7], Extremely Fast Decision Tree (EFDT) [41], and Very Fast Decision Tree (VFDT) [18]. These algorithms were implemented within the Massive Online Analysis (MOA) framework[1], and tuned for each dataset using a grid search strategy, with ensembles of up to 90 components, grace periods set to 100, 400, and 1000, and tie-splitting thresholds configured to 0.01, 0.05, and 0.1 as the hyperparameter settings. The Hoeffding bound confidence level was fixed at 1×10^{-7} across all experiments. Adaptive Naive Bayes prediction was used at the leaf level for all models. For the LTM, we use the publicly available first version of the transformer-based TabPFN[2]. A key practical advantage of LTMs is their minimal reliance on hyperparameter tuning. In TabPFN, the primary source of variability arises from the ensemble prediction mechanism, in which feature orderings and scalings are randomly permuted to account for the permutation invariance of tabular columns. In our experiments, 4 permutations were used. Increasing the number of permutations reduces stochasticity and yields a closer approximation to the Bayesian posterior, typically resulting in monotonic performance improvements. This behavior contrasts with tree-based ensembles, where adding models introduces explicit bias–variance–diversity trade-offs. The proposed dual-memory mechanism also introduces very few parameters. For context optimization, M was set to 600, 800, and 1000 instances, and M_{short} to 0.65, 0.75, 0.85 instances. Moreover, an initial warming period T_{warm} of 100 instances was considered to collect samples into the memory, without making predictions.

In evaluating the application of any foundation model, one can consider both intrinsic and extrinsic evaluation metrics [8]. Extrinsic evaluation refers to measuring the performance of the adapted foundational model on a downstream task, whereas intrinsic evaluation refers to directly measuring the foundational model's quality. Since this work is concerned with the adaptability of LTMs to streams, only extrinsic performance evaluation is performed, using a prequential evaluation strategy, where each instance is first used to test then update the classifier in an online manner (instance by instance) [22]. The used data streams are from the USP data stream repository[3], encompassing both binary and multiclass classification tasks: NOAA Weather (18,159 instances), SmartMeter LADPU

[1] https://github.com/Waikato/moa
[2] https://github.com/PriorLabs/TabPFN
[3] https://sites.google.com/view/uspdsrepository

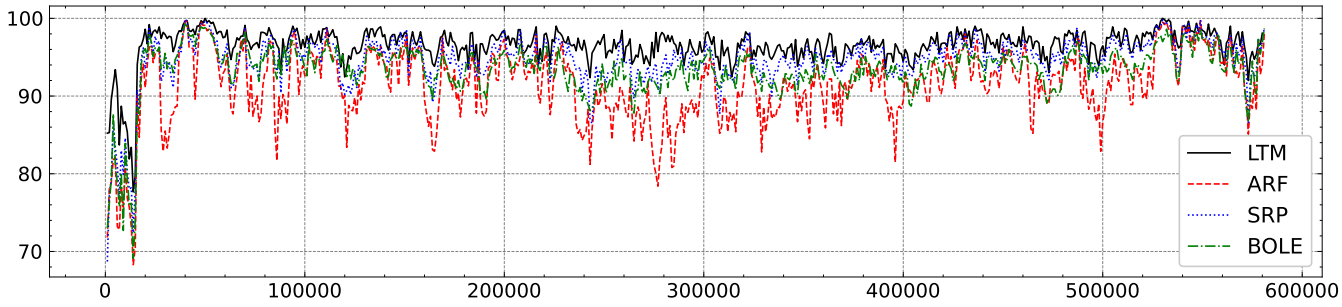


Figure 7: Prequential accuracy over time

(22,950), Electricity (45,312), Rialto Bridge Timelapse (82,250), Posture (164,860), Forest CoverType (581,012) and PokerHand (829,201). Table 1 and Figure 8 compare LTM against all baseline algorithms. LTM achieves the highest overall performance, with an average accuracy of 82.65% and a mean rank of 1. The strongest baselines, ARF and SRP, reach 77.47% and 78.87%, respectively. The largest improvements are observed on the POKER, METER, RIALTO, and NOAA datasets, where LTM exceeds the best alternative by 8.34 pp, 3.93 pp, 3.91 pp, and 2.93 pp, respectively. Two-tailed t-tests ($p < 0.05$) confirm LTM's superiority in all cases except against ARF on POSTURE, BOLE on ELEC/COVER, and SRP on COVER.

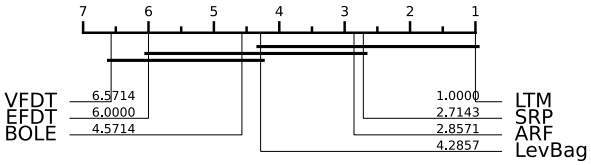


Figure 8: Nemeyi test at p-value = 5%

Grid searches over total memory sizes {600, 800, 1000} and short-term ratios {0.65, 0.75, 0.85} produce highly stable accuracies, with standard deviations up to only 0.31, with the exception of RIALTO. In contrast, several baselines exhibit substantial sensitivity to hyperparameters. Single-tree models such as EFDT and VFDT show standard deviations as high as 4.10 and 7.02, while ensemble methods such as SRP, LevBag, and BOLE can still fluctuate up to 1.81, 5.78, and 1.54, respectively. These results highlight LTM's robustness and consistency across conditions. Table 2 reports the ablation study of each individual component of the dual-memory strategy. Incorporating the dual strategy consistently improves performance over the long-term component alone, with an average gain of +3.24 (range +2.33 to +4.59). These gains are attenuated in datasets with large number of classes and highly volatile class distributions, as reflected by the weak performance of the Majority-Class baseline. The dual strategy also improves over short-term memory alone (+0.31 on average), though gains are smaller and sometimes negative (ELEC, COVER). This is likely attributable to strong temporal dependencies, as suggested by the high accuracy of the No-Change classifier. Increasing context size from 600 to 1000 yields an average improvement of +0.75, statistically significant for most datasets.

Table 2: Ablation study (*: no statistical difference)

Dataset	Dual - Long	Dual - Short	1000 - 600
NOAA	+4.59	+0.89	+0.50*
METER	+3.35	+0.27*	+0.98
ELEC	+4.11	-0.22*	+0.89
RIALTO	+3.45	+0.91	+1.54
POSTURE	+2.33	+0.24*	+0.39*
COVER	+2.48	-0.12*	+0.46*
POKER	+2.35	+0.18*	+0.47
Average	+3.24	+0.31	+0.75

Figure 7 illustrates the evolution of accuracy over time for COVER, showing that LTM experiences significantly smaller performance drops at drift points. While these baselines re-stabilize quickly, LTM's windowing approach excels in handling drifts, maintaining a strong focus on recent instances. In extended intervals of stable concepts, however, the baselines tend to converge toward LTM's accuracy. To further substantiate the results, Table 3 presents the reported accuracies from the original papers of the various baselines, including not only their own performance but also the results of other algorithms: Online Smooth Boosting (OB) [14], Online Accuracy Updated Ensemble (OAUE) [11], Dynamic Weighted Majority (DWM) [33], and Streaming Gradient Boosted Trees (SGBT) [28]. Notably, none report results outperforming LTM.

Table 3: Prequential accuracy reported in other benchmarks

Model	SGBT*/BOLE†		SRP (n=100)		ARF (n=100)	
	ELEC	COVER	ELEC	COVER	ELEC	COVER
ARF	90.62*	94.72*	88.71	94.69	89.67	94.97
LevBag	89.71†	88.13†	90.16	94.86	89.51	95.1
SRP	89.68*	95.34*	88.82	95.25	89.86	95.35
OB	89.51*	92.69*	85.25	90.33	89.52	92.7
OAUE			88.28	90.17	87.41	92.86
DWM	88.52†	87†	87.76	88.29	87.76	88.52
SGBT	88.5*	94.29*				
BOLE	90.04†	90.16†				
LTM	**91.82**	**95.53**	**91.82**	**95.53**	**91.82**	**95.53**

While the performance of LTM is well-established, it is essential to evaluate its adherence to key stream mining requirements [6, 40]: (R1) processing one example at a time and inspecting it only once, (R2) using a limited amount of memory, (R3) operating within a limited time budget, and (R4) being ready to make predictions at any point. Regarding (R1), no strict constraint prevents an algorithm from temporarily storing examples internally. As long as these stored instances are eventually discarded to satisfy (R2), an algorithm remains compliant with this requirement. In terms of (R2), although LTMs may require substantial memory storage, their memory footprint remains constant, enabling them to handle data volumes far exceeding the available working memory. In this regard, memory limitations are a physical constraint that must be balanced with adherence to (R3), i.e., ensuring feasible runtime performance. The memory in LTMs can be divided into two categories: the one allocated for running statistics of long- and short-term memory and the memory used to store the foundational model. While data stream mining algorithms typically optimize memory efficiency by maintaining running statistics that directly inform predictions, LTMs inherently separate these two types of memory. For (R3), LTMs must scale efficiently to any number of examples, which is ensured by its linear runtime complexity concerning the number of processed instances. Furthermore, the computational cost per example is upper-bounded by the number of features, classes and instances within the given context. A critical requirement for real-time operation is that the LTM processes instances as fast as they arrive, as failure to do so leads to data loss. For reference, in MOA, an ARF with 100 hoeffding trees processes 100k samples with 30 features in 2.4 ms per instance. While LTMs may be slower, several strategies can mitigate this, depending on the architecture. For example, Table 4 reports the processing time per sample when using TabPFN on a NVIDIA Tesla T4 GPU to predict ten instances in parallel in a single forward pass. Alternatively, a hypernetwork-based LTM [9] detaches the inference cost of the LTM from that of the prediction by the generated network, suggesting a different strategy. Classic stream management techniques can also be applied on an application-specific basis to ensure timely predictions. For example, instances can be offloaded to other learners or buffered for processing during idle periods. Finally, concerning (R4), while conventional stream learners continuously update their models based on evolving statistics, a LTM translates these statistics into a structured representation, incorporating a selected subset of in-context examples.

Table 4: Processing time per instance

Name	Classes	Features	Time (ms)
NOAA	2	8	18.4
METER	10	96	81.4
ELEC	2	8	16.9
RIALTO	10	27	28.7
POSTURE	11	3	20.3
COVER	7	54	41.1
POKER	10	11	24.2

6 Conclusions

In the early stages of machine learning, the dominant belief was that the best way to solve a task was to meticulously design a specific dataset and tailor a model exclusively for that task. Over time, it became evident that for certain types of unstructured data, such as image recognition, a general-purpose model could be fine-tuned on specific datasets, yielding effective results. Today, we stand at a point where highly versatile models, such as general-purpose transformers, can be prompted to tackle a wide range of tasks without task-specific training. With this work, we embrace this evolving paradigm. So far, we have identified LTMs as a promising candidate for streaming solutions, warranting further research. The key challenge is not merely the size of the model or processing speed but rather the dynamic interplay between data arrival, training, recovery, and inference. As technology advances, models will naturally become smaller, faster, and more affordable. In the meantime, efforts should focus on designing algorithms that align more closely with the core principles of data stream mining: processing data in its natural order, handling labels that may be correct at the time of request but outdated or even incorrect by the time they become available, minimizing interference among concepts, and so on.

Building on this foundation, several promising research directions emerge within the main goal of context optimization in selecting data with properties similar to the desired target distribution. For instance, distribution matching does not need to be constrained to a uniform context across all query points. Instead, a more flexible approach involves storing a larger pool of encoded knowledge that can be dynamically grouped into adaptive local contexts, tailored to each specific query [9]. This retrieval-based strategy allows to go beyond the proposed temporal assumption, where recent points are likely to be of the same concept. By first applying a distribution diversification method to prioritize heterogeneity in a sample and remove redundancies, distribution matching methods can operate within a representation space that allows for similarity to be measured across data points. Thus allowing to preserve valuable information that could be useful in the future and adopt new inductive biases on how to match data distributions, such as: the smoothness assumption, where if two points reside in a high-density region are close, then so should be their corresponding outputs; the cluster assumption, where if points are in the same cluster, they are likely to be of the same concept; the manifold assumption, where high-dimensional data lies in a low-dimensional manifold. For this purpose, online synopsis techniques, such as histograms, wavelets, sketches can be used to incrementally capture the internal structure of the classes [50], and followed by an offline mining process performed on the stored micro-cluster synopsis whenever a LTM sends a request. Further visionary perspectives on these challenges and opportunities were discussed at the *Streaming Continual Learning AAAI Bridge 2026* [39]. The authors also plan to pursue several of these directions and welcome feedback and collaboration.

Acknowledgments

Work funded by Portugal 2030 and the national Foundation for Science and Technology under Ph.D. scholarship PRT/BD/154713/2023, project FOXPM (COMPETE2030-FEDER-00923300 – 14731), and project doi.org/10.54499/UID/00760/2025.

References

[1] Gabriel Jonas Aguiar and Alberto Cano. 2023. Enhancing concept drift detection in drifting and imbalanced data streams through meta-learning. In *2023 IEEE International Conference on Big Data (BigData)*. IEEE, 2648–2657.

[2] Eduardo G Altmann, Janet B Pierrehumbert, and Adilson E Motter. 2009. Beyond word frequency: Bursts, lulls, and scaling in the temporal distributions of words. *PLOS one* 4, 11 (2009), e7678.

[3] Ryan Anderson, Yun Sing Koh, and Gillian Dobbie. 2016. Recurring concept meta-learning for evolving data streams. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 3–14.

[4] Yuntao Bai, Saurav Kadavath, Suhail Kundu, Amanda Askell, Jackson Kernion, Andy Drain, Deep Ganguli, Evan Hubinger, and David Krueger. 2022. Uncovering mesa-optimization algorithms in Transformers. *arXiv preprint arXiv:2211.12316* (2022).

[5] Leonard E Baum and Ted Petrie. 1966. Statistical inference for probabilistic functions of finite state Markov chains. *The annals of mathematical statistics* 37, 6 (1966), 1554–1563.

[6] Albert Bifet and Ricard Gavalda. 2009. Adaptive learning from evolving data streams. In *Advances in Intelligent Data Analysis VIII: 8th International Symposium on Intelligent Data Analysis, IDA 2009, Lyon, France, August 31-September 2, 2009. Proceedings 8*. Springer, 249–260.

[7] Albert Bifet, Geoff Holmes, and Bernhard Pfahringer. 2010. Leveraging bagging for evolving data streams. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2010, Barcelona, Spain, September 20-24, 2010, Proceedings, Part I 21*. Springer, 135–150.

[8] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. 2022. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258* (2022).

[9] David Bonet, Daniel Mas Montserrat, Xavier Giró-i Nieto, and Alexander G Ioannidis. 2024. HyperFast: Instant Classification for Tabular Data. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 11114–11123.

[10] Tom B Brown. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* (2020).

[11] Dariusz Brzezinski and Jerzy Stefanowski. 2014. Combining block-based and online methods in learning ensembles from concept drifting data streams. *Information Sciences* 265 (2014), 50–67.

[12] Stephanie Chan, Adam Santoro, Andrew Lampinen, Jane Wang, Aaditya Singh, Pierre Richemond, James McClelland, and Felix Hill. 2022. Data distributional properties drive emergent in-context learning in transformers. *Advances in Neural Information Processing Systems* 35 (2022), 18878–18891.

[13] Joe Cheal. 2011. What is the opposite of meta? *ANLP Acuity Vol* 2 (2011).

[14] Shang-Tse Chen, Hsuan-Tien Lin, and Chi-Jen Lu. 2012. An online boosting algorithm with theoretical justifications. *arXiv preprint arXiv:1206.6422* (2012).

[15] Roberto Souto Maior de Barros, Silas Garrido T de Carvalho Santos, and Paulo Mauricio Gonçalves Júnior. 2016. A boosting-like online learning ensemble. In *2016 international joint conference on neural networks (IJCNN)*. IEEE, 1871–1878.

[16] Elaine Ribeiro de Faria, André Carlos Ponce de Leon Ferreira Carvalho, and Joao Gama. 2016. MINAS: multiclass learning algorithm for novelty detection in data streams. *Data mining and knowledge discovery* 30 (2016), 640–680.

[17] Tuan Dinh, Yuchen Zeng, Ruisu Zhang, Ziqian Lin, Michael Gira, Shashank Rajput, Jy-yong Sohn, Dimitris Papailiopoulos, and Kangwook Lee. 2022. Lift: Language-interfaced fine-tuning for non-language machine learning tasks. *Advances in Neural Information Processing Systems* 35 (2022), 11763–11784.

[18] Pedro Domingos and Geoff Hulten. 2000. Mining high-speed data streams. In *Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM, 71–80.

[19] Pedro M. Domingos and Geoff Hulten. 2001. Catching up with the Data: Research Issues in Mining Data Streams. In *2001 ACM SIGMOD Workshop on Research Issues in Data Mining and Knowledge Discovery (DMKD '01)*. Santa Barbara, CA, USA.

[20] Xi Fang, Weijie Xu, Fiona Anting Tan, Jiani Zhang, Ziqing Hu, Yanjun Jane Qi, Scott Nickleach, Diego Socolinsky, Srinivasan Sengamedu, Christos Faloutsos, et al. 2024. Large language models (LLMs) on tabular data: Prediction, generation, and understanding-a survey. (2024).

[21] Joao Gama, Pedro Pereira Rodrigues, Eduardo Spinosa, and Andre Carvalho. 2010. Knowledge discovery from data streams. In *Web Intelligence and Security*. IOS Press, 125–138.

[22] Joao Gama, Raquel Sebastiao, and Pedro Pereira Rodrigues. 2013. On evaluating stream learning algorithms. *Machine learning* 90 (2013), 317–346.

[23] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A survey on concept drift adaptation. *ACM computing surveys (CSUR)* 46, 4 (2014), 1–37.

[24] Siddhant Garg, Sivaraman Balakrishnan, Zachary Kolter, and Zachary Lipton. 2022. Transformers can do bayesian inference. *arXiv preprint arXiv:2112.10510* (2022).

[25] Heitor M Gomes, Albert Bifet, Jesse Read, Jean Paul Barddal, Fabrício Enembreck, Bernhard Pfharinger, Geoff Holmes, and Talel Abdessalem. 2017. Adaptive random forests for evolving data stream classification. *Machine Learning* 106 (2017), 1469–1495.

[26] Heitor Murilo Gomes, Jesse Read, and Albert Bifet. 2019. Streaming random patches for evolving data stream classification. In *2019 IEEE international conference on data mining (ICDM)*. IEEE, 240–249.

[27] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. 2021. Revisiting deep learning models for tabular data. *Advances in Neural Information Processing Systems* 34 (2021), 18932–18943.

[28] Nuwan Gunasekara, Bernhard Pfahringer, Heitor Gomes, and Albert Bifet. 2024. Gradient boosted trees for evolving data streams. *Machine Learning* 113, 5 (2024), 3325–3352.

[29] Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. 2023. Tabllm: Few-shot classification of tabular data with large language models. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 5549–5581.

[30] Noah Hollmann, Samuel Müller, Katharina Eggensperger, and Frank Hutter. 2022. Tabpfn: A transformer that solves small tabular classification problems in a second. *arXiv preprint arXiv:2207.01848* (2022).

[31] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. 2025. Accurate predictions on small data with a tabular foundation model. *Nature* 637, 8045 (2025), 319–326.

[32] Evan Hubinger, Chris van Merwijk, Vladimir Mikulik, Joar Skalse, and Scott Garrabrant. 2019. Risks from learned optimization in advanced machine learning systems. *arXiv preprint arXiv:1906.01820* (2019).

[33] J Kotler and Marcus Maloof. 2003. Dynamic weighted majority: A new ensemble method for tracking concept drift. In *IEEE International Conference on Data Mining*. 123–130.

[34] Bartosz Krawczyk, Leandro L Minku, João Gama, Jerzy Stefanowski, and Michał Woźniak. 2017. Ensemble learning for data stream analysis: A survey. *Information Fusion* 37 (2017), 132–156.

[35] Soochan Lee, Jaehyeon Son, and Gunhee Kim. 2024. Recasting continual learning as sequence modeling. *Advances in Neural Information Processing Systems* 36 (2024).

[36] Yingcong Li, Muhammed Emrullah Ildiz, Dimitris Papailiopoulos, and Samet Oymak. 2023. Transformers as algorithms: Generalization and stability in in-context learning. In *International Conference on Machine Learning*. PMLR, 19565–19594.

[37] Peter J Liu, Mohammad Saleh, Etienne Pot, Ben Goodrich, Ryan Sepassi, Lukasz Kaiser, and Noam Shazeer. 2018. Generating wikipedia by summarizing long sequences. *arXiv preprint arXiv:1801.10198* (2018).

[38] Afonso Lourenço, João Rodrigo, João Gama, and Goreti Marreiros. 2025. DFDT: Dynamic Fast Decision Tree for IoT Data Stream Mining on Edge Devices. *arXiv preprint arXiv:2502.14011* (2025).

[39] Afonso Lourenço, João Gama, Eric P Xing, and Goreti Marreiros. 2025. Bridging Streaming Continual Learning via In-Context Large Tabular Models. *arXiv preprint arXiv:2512.11668* (2025).

[40] Afonso Lourenço, João Rodrigo, João Gama, and Goreti Marreiros. 2025. On-device edge learning for IoT data streams: a survey. *arXiv preprint arXiv:2502.17788* (2025).

[41] Chaitanya Manapragada, Geoffrey I Webb, and Mahsa Salehi. 2018. Extremely fast decision tree. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1953–1962.

[42] Vinicius E Martins, Alberto Cano, and Sylvio B Junior. 2023. Meta-learning for dynamic tuning of active learning on stream classification. *Pattern Recognition* (2023).

[43] Colin McDiarmid et al. 1989. On the method of bounded differences. *Surveys in combinatorics* 141, 1 (1989), 148–188.

[44] Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Schuster, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2022. Rethinking the role of demonstrations: What makes in-context learning work? *arXiv preprint arXiv:2202.12837* (2022).

[45] Samuel Müller, Noah Hollmann, Sebastian Pineda Arango, Josif Grabocka, and Frank Hutter. 2021. Transformers can do bayesian inference. *arXiv preprint arXiv:2112.10510* (2021).

[46] Thomas Nagler and Jesper Lützen. 2023. Statistical Foundations of Prior-Data Fitted Networks. *arXiv preprint arXiv:2305.11175* (2023).

[47] Ismail Nejjar, Faez Ahmed, and Olga Fink. 2024. IM-Context: In-Context Learning for Imbalanced Regression Tasks. *arXiv preprint arXiv:2405.18202* (2024).

[48] Lara Neves, Afonso Lourenço, Alberto Cano, and Goreti Marreiros. 2025. *Online Hierarchical Partitioning of the Output Space in Extreme Multi-Label Data Streams*. IOS Press. doi:10.3233/faia250975

[49] Jesse Read. 2023. From multi-label learning to cross-domain transfer: A model-agnostic approach. *Applied Intelligence* 53, 21 (2023), 25135–25153.

[50] Jonathan A Silva, Elaine R Faria, Rodrigo C Barros, Eduardo R Hruschka, André CPLF De Carvalho, and João Gama. 2013. Data stream clustering: A survey. *ACM Computing Surveys (CSUR)* 46, 1 (2013), 1–31.

[51] Gowthami Somepalli, Micah Goldblum, Avi Schwarzschild, C Bayan Bruss, and Tom Goldstein. 2021. SAINT: Improved neural networks for tabular data via row attention and contrastive pre-training. *arXiv preprint arXiv:2106.01342* (2021).

[52] Avijit Thawani, Jay Pujara, Pedro A Szekely, and Filip Ilievski. 2021. Representing numbers in NLP: a survey and a vision. *arXiv preprint arXiv:2103.13136* (2021).

[53] Boris van Breugel, Jonathan Crabbé, Rob Davis, and Mihaela van der Schaar. 2024. LaTable: Towards Large Tabular Models. *arXiv preprint arXiv:2406.17673* (2024).

[54] Boris van Breugel and Mihaela van der Schaar. 2024. Why tabular foundation models should be a research priority. *arXiv preprint arXiv:2405.01147* (2024).

[55] Jan N van Rijn, Geoffrey Holmes, Bernhard Pfahringer, and Joaquin Vanschoren. 2014. Algorithm selection on data streams. In *Discovery Science: 17th International Conference, DS 2014, Bled, Slovenia, October 8-10, 2014. Proceedings 17*. Springer, 325–336.

[56] Jan N van Rijn, Geoffrey Holmes, Bernhard Pfahringer, and Joaquin Vanschoren. 2015. Having a blast: Meta-learning and heterogeneous ensembles for data streams. In *2015 ieee international conference on data mining*. IEEE, 1003–1008.

[57] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[58] Julius von Kügelgen, Yash Sharma, Luigi Gresele, Wieland Brendel, Bernhard Schölkopf, and Michel Besserve. 2022. Transformers learn in-context by gradient descent. *arXiv preprint arXiv:2212.07677* (2022).

[59] Julius von Kügelgen, Yash Sharma, Luigi Gresele, Wieland Brendel, Bernhard Schölkopf, and Michel Besserve. 2022. What learning algorithm is in-context learning? Investigations with linear models. *arXiv preprint arXiv:2211.15661* (2022).

[60] Zifeng Wang and Jimeng Sun. 2022. Transtab: Learning transferable tabular transformers across tables. *Advances in Neural Information Processing Systems* 35 (2022), 2902–2915.

[61] Chuhan Wu, Fangzhao Wu, Tao Qi, Yongfeng Huang, and Xing Xie. 2021. Fastformer: Additive attention can be all you need. *arXiv preprint arXiv:2108.09084* (2021).

[62] Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. 2022. An explanation of in-context learning as implicit bayesian inference. *arXiv preprint arXiv:2111.02080* (2022).

[63] Shuo Xu and Jing Wang. 2016. Mining recurring concept drifts with limited labeled streaming data. *ACM Transactions on Intelligent Systems and Technology (TIST)* 7, 1 (2016), 1–32.

[64] Han-Jia Ye, Qi-Le Zhou, and De-Chuan Zhan. 2023. Training-free generalization on heterogeneous tabular data via meta-representation. *arXiv preprint arXiv:2311.00055* (2023).

[65] Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. 2021. Calibrate before use: Improving few-shot performance of language models. In *International conference on machine learning*. PMLR, 12697–12706.

[66] Bingzhao Zhu, Xingjian Shi, Nick Erickson, Mu Li, George Karypis, and Mahsa Shoaran. 2023. Xtab: Cross-table pretraining for tabular transformers. *arXiv preprint arXiv:2305.06090* (2023).